

International Handbooks on Information Systems

Jacek Blazewicz
Klaus H. Ecker
Erwin Pesch
Günter Schmidt
Malgorzata Sterna
Jan Weglarz

Handbook on Scheduling

From Theory to Practice

Second Edition



Springer

International Handbooks on Information Systems

Series editors

Peter Bernus
Jacek Blazewicz
Günter Schmidt
Michael J. Shaw

International Handbooks on Information Systems is a series of volumes offering state-of-the-art surveys on selected topics of information theory and applications.

The scope of the series comprises topics such as architectures of information systems, enterprise architecture, data management in information systems, electronic commerce, knowledge management, ontologies and others. To date more than a dozen volumes have been published, some of them in multiple editions.

The objective is to involve contributors who are the authoritative figures in this area of information systems in each volume in order to provide a reference source for problem solvers in business, industry, and government, which can also be used by professional researchers and graduate students.

To this end, a distinguished international group of academics and practitioners will be invited to contribute articles on their respective fields of expertise.

More information about this series at <http://www.springer.com/series/3795>

Jacek Blazewicz • Klaus H. Ecker • Erwin Pesch
Günter Schmidt • Malgorzata Sterna • Jan Weglarz

Handbook on Scheduling

From Theory to Practice

Second Edition



Springer

Jacek Blazewicz
Institute of Computing Science
Poznan University of Technology
Poznan, Poland

Institute of Bioorganic Chemistry
Polish Academy of Sciences
Poznan, Poland

European Centre for Bioinformatics
and Genomics
Poznan, Poland

Erwin Pesch
Faculty III - School of Economic Disciplines
University of Siegen
Siegen, Germany
HHL - Leipzig Graduate School of Management
Center for Advanced Studies in Management
Leipzig, Germany

Malgorzata Sterna
Institute of Computing Science
Poznan University of Technology
Poznan, Poland

Klaus H. Ecker
Institute of Informatics
Clausthal University of Technology
Clausthal-Zellerfeld, Germany

Günter Schmidt
Saarland University
Saarbrücken, Germany

University of Cape Town
Cape Town, South Africa

Jan Weglarz
Institute of Computing Science
Poznan University of Technology
Poznan, Poland

Poznan Supercomputing
and Networking Center
Poznan, Poland

International Handbooks on Information Systems
ISBN 978-3-319-99848-0 ISBN 978-3-319-99849-7 (eBook)
<https://doi.org/10.1007/978-3-319-99849-7>

Library of Congress Control Number: 2018956311

© Springer Nature Switzerland AG 2019

This work is subject to copyright. All rights are reserved by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, express or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

FOREWORD

This handbook¹ is a continuation of [1] which has received kind acceptance of a wide readership, and, as [1], is the result of a long lasting German-Polish cooperation. Its specificity remains the same as it underlines a transition from theory to practice in a wide spectrum of scheduling problems.

However, realizing the suggestion of the Publisher to prepare a new version of the handbook, we decided to publish a new textbook rather than only a new edition of the previous one. This followed from our conviction based on many discussions that some important new subjects should be included. They are characterized in Chapter 1. We also decided to enlarge the group of authors by including Małgorzata Sterna.

We very much hope that in this way the handbook will be of interest to even a wider audience than the previous one.

During the preparation of the book many colleagues have discussed with us the different topics presented in it. We are not able to list all of them but we would like to express our gratitude towards Nils Boysen, Nadia Brauner, Edmund Burke, Ulrich Dorndorf, Maciej Drozdowski, Toan Phan Huy, Florian Jaehn, Joanna Józefowska, Imed Kacem, Graham Kendall, Misha Kovalyov, Dominik Kress, Wiesław Kubiak, Sebastian Meiswinkel, Jenny Nossack, Ceyda Oğuz, Alena Otto, Denis Trystram and Dominique deWerra. Separate thanks are due to Stanisław Gawiejniewicz who revised references and prepared Chapter 12.

References

- [1] J. Błażewicz, K. Ecker, E. Pesch, G. Schmidt, J. Węglarz, *Handbook on Scheduling - From Theory to Applications*, Springer Verlag, Berlin, 2007.

¹ This work was partially supported by PUT grant No 09/91/DSPB/0649.

Contents

1	Introduction	1
	References	9
2	Basics	11
2.1	Sets and Relations	11
2.2	Problems, Algorithms, Complexity	13
2.2.1	Problems and Their Encoding	13
2.2.2	Algorithms	15
2.2.3	Complexity	18
2.3	Graphs and Networks	23
2.3.1	Basic Notions	23
2.3.2	Special Classes of Digraphs	24
2.3.3	Networks	27
2.4	Enumerative Methods	34
2.4.1	Dynamic Programming	35
2.4.2	Branch and Bound	35
2.5	Heuristic and Approximation Algorithms	37
2.5.1	Approximation Algorithms	37
2.5.2	Local Search Heuristics	41
	References	55
3	Definition, Analysis and Classification of Scheduling Problems	61
3.1	Definition of Scheduling Problems	61
3.2	Analysis of Scheduling Problems and Algorithms	66
3.3	Motivations for Deterministic Scheduling Problems	69
3.4	Classification of Deterministic Scheduling Problems	72
	References	75
4	Scheduling on One Processor	77
4.1	Minimizing Schedule Length	77
4.1.1	Scheduling with Release Times and Deadlines	78
4.1.2	Scheduling with Release Times and Delivery Times	85
4.2	Minimizing Mean Weighted Flow Time	87

4.3	Minimizing Due Date Involving Criteria	99
4.3.1	Maximum Lateness	100
4.3.2	Number of Tardy Tasks	108
4.3.3	Mean Tardiness	113
4.3.4	Mean Earliness	117
4.4	Minimizing Change-Over Cost	118
4.4.1	Setup Scheduling	118
4.4.2	Lot Size Scheduling	121
4.5	Other Criteria	126
4.5.1	Maximum Cost	126
4.5.2	Total Cost	131
	References	134
5	Scheduling on Parallel Processors	141
5.1	Minimizing Schedule Length	141
5.1.1	Identical Processors	141
5.1.2	Uniform and Unrelated Processors	162
5.2	Minimizing Mean Flow Time	172
5.2.1	Identical Processors	172
5.2.2	Uniform and Unrelated Processors	173
5.3	Minimizing Due Date Involving Criteria	177
5.3.1	Identical Processors	177
5.3.2	Uniform and Unrelated Processors	183
5.4	Lot Size Scheduling	186
	References	191
6	Communication Delays and Multiprocessor Tasks	199
6.1	Introductory Remarks	199
6.2	Scheduling Multiprocessor Tasks	205
6.2.1	Parallel Processors	205
6.2.2	Dedicated Processors	213
6.2.3	Refinement Scheduling	219
6.3	Scheduling Uniprocessor Tasks with Communication Delays	221
6.3.1	Scheduling without Task Duplication	222
6.3.2	Scheduling with Task Duplication	225
6.3.3	Scheduling in Processor Networks	226
6.4	Scheduling Divisible Tasks	228
	References	236

7	Scheduling in Hard Real-Time Systems	243
7.1	Introduction	243
7.1.1	What is a Real-Time System?	244
7.1.2	Examples of Real-Time Systems	245
7.1.3	Characteristics of Real-Time Systems	246
7.1.4	Functional Requirements for Real-Time Systems	247
7.2	Basic Notions	248
7.2.1	Structure of a Real-Time System	248
7.2.2	The Task Model	249
7.2.3	Schedules	250
7.3	Single Processor Scheduling	252
7.3.1	Static Priority Scheduling	253
7.3.2	Dynamic Priority Scheduling	262
7.4	Scheduling Periodic Tasks on Parallel Processors	264
7.5	Resources	265
7.6	Variations of the Periodic Task Model	266
	References	267
8	Flow Shop Scheduling	271
8.1	Introduction	271
8.1.1	The Flow Shop Scheduling Problem	271
8.1.2	Complexity	273
8.2	Exact Methods	274
8.2.1	The Algorithms of Johnson and Akers	274
8.2.2	Dominance and Branching Rules	277
8.2.3	Lower Bounds	278
8.3	Approximation Algorithms	282
8.3.1	Priority Rule and Local Search Based Heuristics	282
8.3.2	Worst-Case Analysis	285
8.3.3	No Wait in Process	289
8.4	Scheduling Flexible Flow Shops	291
8.4.1	Problem Formulation	291
8.4.2	Heuristics and Their Performance	294
8.4.3	A Model	296
8.4.4	The Makespan Minimization Problem	297
8.4.5	The Mean Flow Time Problem	311
	References	316

9	Open Shop Scheduling	321
9.1	Complexity Results	321
9.2	A Branch and Bound Algorithm for Open Shop Scheduling	323
9.2.1	The Disjunctive Model of the OSP	323
9.2.2	Constraint Propagation and the OSP	326
9.2.3	The Algorithm and Its Performance	332
	References	341
10	Scheduling in Job Shops	345
10.1	Introduction	345
10.1.1	The Problem	345
10.1.2	Modeling	345
10.1.3	Complexity	353
10.1.4	The History	354
10.2	Exact Methods	357
10.2.1	Branch and Bound	358
10.2.2	Lower Bounds	358
10.2.3	Branching	360
10.2.4	Valid Inequalities	363
10.3	Approximation Algorithms	365
10.3.1	Priority Rules	365
10.3.2	The Shifting Bottleneck Heuristic	367
10.3.3	Opportunistic Scheduling	370
10.3.4	Local Search	372
10.4	Conclusions	392
	References	393
11	Scheduling with Limited Processor Availability	403
11.1	Problem Definition	404
11.2	One Machine Problems	407
11.3	Parallel Machine Problems	409
11.3.1	Minimizing the Sum of Completion Times	409
11.3.2	Minimizing the Makespan	410
11.3.3	Dealing with Due Date Involving Criteria	418
11.4	Shop Problems	420
11.4.1	Flow Shop Problems	420
11.4.2	Open Shop Problems	423
11.5	Conclusions	423
	References	425

12 Time-Dependent Scheduling	431
12.1 Introduction	432
12.2 Forms of Time-Dependent Processing Times	433
12.2.1 General Forms	433
12.2.2 Special Forms	434
12.3 One Machine Problems	436
12.3.1 Proportionally Deteriorating Processing Times	437
12.3.2 Proportional-Linearly Deteriorating Processing Times	438
12.3.3 Proportional-Linearly Shortening Processing Times	439
12.3.4 Linearly Deteriorating Processing Times	440
12.3.5 Linearly Shortening Processing Times	446
12.3.6 Non-Linearly Deteriorating Processing Times	447
12.3.7 Other One Machine Problems	448
12.4 Parallel Machine Problems	457
12.4.1 Proportionally Deteriorating Processing Times	457
12.4.2 Linearly Deteriorating Processing Times	462
12.4.3 Non-Linearly Deteriorating Processing Times	462
12.5 Dedicated Machine Problems	463
12.5.1 Proportionally Deteriorating Processing Times	463
12.5.2 Proportional-Linearly Deteriorating Processing Times	465
12.5.3 Linearly Deteriorating Processing Times	465
12.5.4 Non-Linearly Deteriorating Processing Times	466
References	467
13 Scheduling under Resource Constraints	475
13.1 Classical Model	475
13.2 Scheduling Multiprocessor Tasks	486
13.3 Scheduling with Continuous Resources	500
13.3.1 Introductory Remarks	501
13.3.2 Processing Speed vs. Resource Amount Model	502
13.3.3 Processing Time vs. Resource Amount Model	511
13.3.4 Ready Time vs. Resource Amount Model	517
References	521
14 Scheduling Imprecise Computations	527
14.1 Introduction	527
14.2 Imprecise Computation Model	530

14.3	Late Work Model.....	532
14.3.1	Single Processor Problems	533
14.3.2	Parallel Processor Problems	545
14.3.3	Dedicated Processor Problems.....	550
14.4	Related Problems	566
14.5	Conclusions	569
	References	570
15	Online Scheduling	577
15.1	Online Scheduling Models	578
15.2	Online Scheduling Algorithms	580
15.3	Competitive Analysis	581
15.3.1	Competitive Ratio	582
15.3.2	Lower Bound	586
15.3.3	Adversary Method	586
15.4	Other Online Scheduling Models	590
15.4.1	Semi-Online Scheduling	590
15.4.2	Online Scheduling with Advice.....	600
15.4.3	Resource Augmentation	601
15.5	Conclusions	602
	References	602
16	Constraint Programming and Disjunctive Scheduling	609
16.1	Introduction	609
16.2	Constraint Satisfaction	611
16.2.1	The Constraint Satisfaction and Optimization Problem...	611
16.2.2	Constraint Propagation	613
16.3	The Disjunctive Scheduling Problem	625
16.3.1	The Disjunctive Model	626
16.3.2	Solution Methods for the DSP	629
16.4	Constraint Propagation and the DSP	629
16.4.1	Some Basic Definitions	630
16.4.2	Precedence Consistency Tests	632
16.4.3	Lower-Level Bound-Consistency	632
16.4.4	Input/Output Consistency Tests.....	641
16.4.5	Input/Output Negation Consistency Tests.....	648
16.4.6	Input-or-Output Consistency Tests.....	654
16.4.7	Energetic Reasoning	655
16.4.8	Shaving	659

16.4.9	A Comparison of Disjunctive Consistency Tests	660
16.4.10	Precedence vs. Disjunctive Consistency Tests	662
16.5	Conclusions	662
16.6	Appendix: Bound Consistency Revisited	663
	References	667
17	Scheduling in Flexible Manufacturing Systems	671
17.1	Introductory Remarks	671
17.2	Scheduling Dynamic Job Shops	674
17.2.1	Introductory Remarks	674
17.2.2	Heuristic Algorithm for the Static Problem	675
17.2.3	Computational Experiments	681
17.3	Simultaneous Scheduling and Routing in some FMS	682
17.3.1	Problem Formulation	682
17.3.2	Vehicle Scheduling for a Fixed Production Schedule	684
17.3.3	Simultaneous Job and Vehicle Scheduling	689
17.4	Batch Scheduling in Flexible Flow Shops under Resource Constraints	691
17.4.1	Introduction - Statement of the Problem	691
17.4.2	Mathematical Formulation	692
17.4.3	Heuristic Solution Approach	702
	References	709
18	Computer Integrated Production Scheduling	713
18.1	Scheduling in Computer Integrated Manufacturing	714
18.2	A Reference Model for Production Scheduling	719
18.3	IPS: An Intelligent Production Scheduling System	727
18.3.1	Interactive Scheduling	734
18.3.2	Knowledge-based Scheduling	749
18.3.3	Integrated Problem Solving	754
	References	758
19	Scheduling in Logistics	761
19.1	Introduction	761
19.2	Vehicle Routing Problem	762
19.3	Concrete Delivery Problem	767
19.3.1	Overview	767
19.3.2	Modeling the Concrete Delivery Problem	768

19.3.3 Related Models	776
19.4 Flight Gate Scheduling Problem	779
19.4.1 Overview	780
19.4.2 Modeling the Flight Gate Scheduling Problem	782
19.4.3 Related Models	789
19.5 Berth and Quay Crane Allocation Problem	792
19.5.1 Overview	793
19.5.2 Modeling the Berth and Quay Crane Allocation Problem ..	794
19.5.3 Related Models	800
19.6 Conclusions	802
References	802
Index	813



1 Introduction

Scheduling problems can be understood in general as the problems of allocating resources over time to perform a set of tasks being parts of some processes, among which computational and manufacturing ones are most important. Tasks individually compete for resources which can be of a very different nature, e.g. manpower, money, processors (machines), energy, tools. The same is true for task characteristics, e.g. ready times, due dates, relative urgency weights, functions describing task processing in relation to allotted resources. Moreover, a structure of a set of tasks, reflecting relations among them, can be defined in different ways. In addition, different criteria which measure the quality of the performance of a set of tasks can be taken into account.

It is easy to imagine that scheduling problems understood so generally appear almost everywhere in real-world situations. Of course, there are many aspects concerning approaches for modeling and solving these problems which are of general methodological importance. On the other hand, however, some classes of scheduling problems have their own specificity which should be taken into account. Since it is rather impossible to treat all these classes with the same attention in a framework of one book, some constraints must be put on the subject area considered. In the case of this handbook these constraints are as follows.

First of all we focus on the problems motivated by applications from industry and service operations management as well as from case studies of real - life problems. Among others there is a detailed description of optimization procedures for acrylic-glass production and the production of helicopter parts in a flexible manufacturing system. We will describe the backbone of an efficient decision support system for airport gate scheduling as well as a flexible flow shop scheduling system in order to manufacture screws and car power brakes.

Second, we deal with deterministic scheduling problems (cf. [Bak74, BCSW86, Bru07, BT09, CCLL95, CMM67, Cof76, Eck77, Fre82, Gaw08, GK87, Len77, Leu04, LLR+93, Pin16, Rin76, RV09, TB06, TGS94, TSS94]), i.e. those in which no variable with a non-deterministic (e.g. probabilistic) description appears. Let us stress that this assumption does not necessarily mean that we deal only with static problems in which all characteristics of a set of tasks and a set of resources are known in advance. We consider also dynamic problems in which some parameters such as task ready times are unknown in advance, and we do not assume any a priori knowledge about them; this approach is even more realistic in many practical situations.

Third, we consider problems in which a set of resources always contains processors (machines). This means that we take into account the specificity of these particular resources in modeling and solving corresponding scheduling problems, but it does not mean that all presented methods and approaches are restricted to this specificity only. The main reason for which we differentiate

processors (we even do not call them "resources" for the convenience of a reader) is that we like to expose especially two broad (and not exclusive) areas of practical applications of the considered problems, namely computer and manufacturing systems.

After the explanation of the handbook's title, we can pass to the description of some of its deeper specificities. They can be meant as compromise we accepted in multi-objective decision situations we had to deal with before and during the preparation of the text. At the beginning, we found a compromise between algorithmic (rather quantitative) and knowledge-based (rather qualitative) approaches to scheduling. We decided to present in the first chapters the algorithmic approach, and at the end to show how it can be integrated with the approach coming from the area of Artificial Intelligence, in order to create a pretty general and efficient tool for solving a broad class of practical problems. In this way we also hopefully found a compromise between rather more computer and rather more manufacturing oriented audience.

The second compromise was concerned with the way of presenting algorithms: formal or descriptive. Basically we decided to adopt a Pascal-like notation, although we allowed for few exceptions in cases where such a presentation would be not efficient.

Next we agreed that the presented material should realize a reasonable compromise between the needs of readers coming from different areas, starting from those who are beginners in the field of scheduling and related topics, and ending with specialists in the area. Thus we included some preliminaries concerning basic notions from discrete mathematics (problems, algorithms and methods), as well as, besides the most recent, also the most important classical results.

Summing up the above compromises, we think that the handbook can be addressed to a quite broad audience, including practitioners and researchers interested in scheduling, and also to graduate or advanced undergraduate students in computer science/engineering, operations research, industrial engineering, management science, business administration, information systems, and applied mathematics curricula.

Finally, we present briefly the outline of the handbook.

In **Chapter 2** basic definitions and concepts used throughout the book are introduced. One of the main issues studied here is the complexity analysis of combinatorial problems. As a unified framework for this presentation the concept of a combinatorial search problem is used. Such notions as: decision and optimization problems, their encoding schemes, input length, complexity classes of problems, are discussed and illustrated by several examples. Since the majority of scheduling problems are computationally hard, two general approaches dealing with such problems are briefly discussed: enumerative and heuristic. First, general enumerative approaches, i.e. dynamic programming and branch and bound are shortly presented. Second, heuristic algorithms are introduced and the ways of analysis of their accuracy in the worst case and on the average are described. Then, we introduce the ideas of general local search metaheuristics known under names: simulated annealing, tabu search, and ejection chains as

well as genetic algorithms. In contrast with previous approaches (e.g. hill-climbing) to deal with combinatorially explosive search spaces about which little knowledge is known a priori, the above mentioned metaheuristics, in combination with problem specific knowledge, are able to escape a local optimum. Basically, the only thing that matters, is the definition of a neighborhood structure in order to step from a current solution to a next, probably better one. The neighborhood structure has to be defined within the setting of the specific problem and with respect to the accumulated knowledge of the previous search process. Furthermore, frequently some random elements guarantee a satisfactory search diversification over the solution space. During the last years the metaheuristics turned out to be very successful in the scheduling area; specific applications will be described in Chapters 8, 9, 10 and 17.

The chapter is complemented by a presentation of the notions from sets and relations, as well as graphs and networks, which will be used in the later chapters.

In **Chapter 3** definitions, assumptions and motivations for deterministic scheduling problems are introduced. We start with the set of tasks, the set of processors (machines) and the set of resources, and with two basic models in deterministic scheduling, i.e. parallel processors and dedicated processors. Then, schedules and their performance measures (optimality criteria) are described. After this introduction, possible ways of analyzing scheduling problems are described with a special emphasis put to solution strategies of computationally hard problems. Finally, motivations for the use of the deterministic scheduling model as well as an interpretation of results, are discussed. Two major areas of applications, i.e. computer and manufacturing systems are especially taken into account. These considerations are complemented by a description of a classification scheme which enables one to present deterministic scheduling problems in a short and elegant way.

Chapter 4 deals with single-processor scheduling. The results given here are mainly concerned with polynomial time optimization algorithms. Their presentation is divided into several sections taking into account especially the following optimality criteria: schedule length, mean (and mean weighted) flow time, and due date involving criteria, such as maximum lateness, number of tardy tasks, mean (and mean weighted) tardiness and a combination of earliness and lateness. In each case polynomial time optimization algorithms are presented, taking into account problem parameters such as the type of precedence constraints, possibility of task preemption, processing and arrival times of tasks, etc. These basic results are complemented by some more advanced ones which take into account change-over cost, also called lot size scheduling, and more general cost functions. Let us stress that in this chapter we present a more detailed classification of subcases as compared to the following chapters. This follows from the fact that the algorithms for the single-processor model are useful also in more complicated cases, whenever a proper decomposition of the latter is carried out. On the other hand, its relative easiness makes it possible to solve optimally in polynomial time many more subcases than in the case of multiple processors.

Chapter 5 carries on an analysis of scheduling problems where multiple parallel processors are involved. As in Chapter 4, a presentation of the results is divided into several subsections depending mainly on the criterion considered and then on problem parameters. Three main criteria are analyzed: schedule length, mean flow time and maximum lateness. A further division of the presented results takes in particular into account the type of processors considered, i.e. identical, uniform or unrelated processors, and then parameters of a set of tasks. Here, scheduling problems are more complicated than in Chapter 4, so not as many optimization polynomial time algorithms are available as before. Hence, more attention is paid to the presentation of polynomial time heuristic algorithms with guaranteed accuracy, as well as to the description of some enumerative algorithms.

In **Chapter 6** new scheduling problems arising in the context of rapidly developing manufacturing as well as parallel computer systems, are considered. When formulating scheduling problems in such systems, one must take into account the fact that some tasks have to be processed on more than one processor at a time. On the other hand, communication issues must be also taken into account in systems where tasks (program modules) are assigned to different processors and exchange information between each other. In the chapter three models are discussed in a sequel. The first model assumes that each so-called multiprocessor task may require more than one processor at a time and communication times are implicitly included in tasks' processing times. The second model assumes that uniprocessor tasks, each assigned to one processor, communicate explicitly via directed links of the task graph. More precise approaches distinguish between coarse grain and fine grain parallelism and discuss their impact on communication delays. Furthermore, task duplication often leads to shorter schedules; this is in particular the case if the communication times are large compared to the processing times. The last model is a combination of the first two models and involves the so called divisible tasks.

Chapter 7 deals with another type of scheduling problems where the tasks are periodic in the sense that they are processed repeatedly and with given frequencies. Particularly in real-time systems designed for controlling some technical facility we are confronted with problems where sets of periodic tasks are to be processed on a single processor or on a distributed or parallel processor system. The chapter starts with a short introduction to real-time systems and discusses characteristic properties and general functional requirements of such systems. Then strategies for scheduling sets of periodic tasks on a single processor and on a multiprocessor system are presented, and the classical results for the rate monotonic and earliest deadline scheduling strategies are discussed from their properties and performance points of view. Another important issue regards runtime problems that appear if tasks use of non-preemptable (non-withdrawable) resources. Finally, several variants of the periodic task model allowing higher flexibility as compared to the simple periodic task model are presented.

In **Chapter 8** flow shop scheduling problems are described, i.e. scheduling a set of jobs (composed of tasks) in shops with a product machine layout. Thus, the jobs have the same manufacturing order. Recent local search heuristics as well as heuristics relying on the two-machine flow shop scheduling problem - which can easily be solved optimally - are considered. Some special flow shops are introduced, e.g. permutation and no-wait ones. The hybrid or flexible flow shop problem is a generalization of the flow shop in such a way that every job can be processed by one among several machines on each machine stage. In recent years a number of effective exact methods have been developed. A major reason for this progress is the development of new job and machine based lower bounds as well as the rapidly increasing importance of constraint programming. We provide a comprehensive and uniform overview on exact solution methods for flexible flow shops with branching, bounding and propagation of constraints, under two different objective functions: minimizing the makespan of a schedule and the mean flow time. For some simple cases we present heuristics with known worst case performance and then describe a branch and bound algorithm for the general case.

In **Chapter 9** we consider the open shop problem where jobs without any precedence constraints are supposed to be scheduled. Only few exact solution methods are available and we motivate our presentation with a description of optimal results for small open shop scheduling problems. We continue describing a branch-and-bound algorithm for solving this problem which performs better than any other existing algorithm. The key to the efficiency of the algorithm lies in the following approach: instead of analyzing and improving the search strategies for finding solutions, the focus is on constraint propagation based methods for reducing the search space. For the first time, many problem instances are solved to optimality in a short amount of computation time.

In **Chapter 10** job shop scheduling problems are investigated. This is the most general form of manufacturing a set of different jobs on a set of machines where each job is characterized by its specific machine order reflecting the jobs production process. We introduce the commonly used representation of the job shop scheduling problems: the disjunctive graph, and its efficient machine representation: the graph matrix, which can be generalized for any graph. The most successful branch and bound ideas are described and we will see that their branching structure is reflected in the neighborhood definitions of many local search methods. In particular tabu search, ejection chains, genetic algorithms as well as the propagation of constraints - this is closely related to the generation of valid inequalities - turned out to become the currently most powerful solution approaches. Moreover, we introduce priority rule based scheduling and describe a well-known opportunistic approach: the shifting bottleneck procedure.

Chapter 11 deals with scheduling problems where the availabilities of processors to process tasks are limited. In the preceding chapters the basic model assumes that all machines are continuously available for processing throughout the planning horizon. This assumption might be justified in some cases but it

does not apply if certain maintenance requirements, breakdowns or other constraints that cause the machines not to be available for processing have to be considered. In this chapter we generalize the basic model in this direction and discuss results related to one machine, parallel machines, and shop scheduling problems where machines are not continuously available for processing.

In **Chapter 12** we are focused on time-dependent scheduling problems, where job processing times are functions of the job starting times. Problems of this kind compose the first group of scheduling problems with variable job processing times discussed in the handbook. Another group of such problems, with resource-dependent job processing times, is discussed in Chapter 13. We begin this chapter with a short description of basics of time-dependent scheduling and the main forms of time-dependent job processing times. Next, we review the most important results of time-dependent scheduling, diving them into groups with respect to machine environment and job processing times form. Whenever it is possible, we illustrate our presentation by examples. As in the whole handbook, we discuss only deterministic algorithms and solution methods. Apart time complexity and algorithms for time-dependent scheduling problems, we also discuss two-agent time-dependent scheduling, bi-criteria time-dependent scheduling, properties of mutually related pairs of time-dependent scheduling problems and applications of matrix methods in time-dependent scheduling.

Chapter 13 deals with resource constrained scheduling. In the first two sections it is assumed that tasks require for their processing processors and certain fixed amounts of discrete resources. The first section presents the classical model where schedule length is to be minimized. In this context several polynomial time optimization algorithms are described. In the next section this model is generalized to cover also the case of multiprocessor tasks. Two algorithms are presented that minimize schedule length for preemptable tasks under different assumptions concerning resource requirements. The last section deals with problems in which additional resources are continuous, i.e. continuously-divisible. We study three classes of scheduling problems of that type. The first one contains problems with parallel processors and tasks described by continuous functions relating their processing speeds to the resource amount allotted at a time. The next two classes are concerned with single processor problems where task processing times or ready times, respectively, are continuous functions of the allotted resource amount.

Chapter 14 is devoted to a certain scheduling model - the imprecise computation model, inspired by practical applications arising in the hard real time environment, introduced in Chapter 7. It allows trading off accuracy of computations in favor of meeting deadlines imposed on tasks. In the imprecise computation model tasks are composed of two subtasks: mandatory and optional ones. The mandatory subtask has to be completed before the deadline in order to obtain a feasible solution, but the optional subtask can be late or even left unfinished. The mandatory subtask corresponds to producing a usable but approximate result with the error modeled by the late part of the optional subtask. Completing both subtasks on time corresponds to precise result with no error. The chapter starts

with exemplary applications which motivated such scheduling problems. Then, we briefly present the general imprecise computation model and we pass, in the next section, to its special case - the late work model, which assumes that tasks consist of optional subtasks only. We present results related to single, parallel and dedicated processors. Particularly, we show a few examples of classical solution techniques introduced in Chapter 2, namely: the dynamic programming and the polynomial time approximation scheme being a family of approximation algorithms. Discussing late work problems, we refer to various topics of the scheduling theory involving for example controllable processing times, time-dependent processing times (i.e. scheduling with learning effect) or multi-agent scheduling. In the last section, some related models inspired by the imprecise computations, as well as mirror scheduling problems are mentioned.

The handbook focuses in general on deterministic scheduling models, which assume that the complete knowledge of a problem instance is provided to a decision maker, i.e. to an algorithm, in advance. **Chapter 15** introduces the fundamentals of online scheduling. The online scheduling can be considered as scheduling with incomplete information, because the decisions on executing tasks are made without knowing a complete instance of the problem, i.e. the input is being revealed to a decision maker piece-by-piece. Online scheduling models can be considered as a bridge between deterministic scheduling and stochastic scheduling, which copes with a problem input given as random variables with certain probability distributions. Online algorithms compute partial schedules whenever a new piece of information requests taking an action from them, i.e. the decisions are made without full knowledge of the future. We present basic online scheduling paradigms, such as online-over-list and online-over-time, which assume that tasks appear in the system in a given sequence or at a given time moment. We show the differences between clairvoyant and non-clairvoyant scheduling models, which reveal various amount of information on incoming tasks. Moreover, we adjust the idea of precedence constraints and preemptions, introduced in Chapter 3 for offline mode, to online mode. We present the general idea of deterministic and randomized online algorithms and describe the techniques used for evaluating their efficiency, based on the competitive analysis and on the lower bound analysis involving the adversary method. These basic ideas are illustrated with a few examples. Moreover, we introduce some enhanced online scheduling models, such as semi-online scheduling, online scheduling with advice or online scheduling with resource augmentation.

Constraint propagation is the central topic of **Chapter 16**. It is an elementary method for reducing the search space of combinatorial search and optimization problems which has become more and more important in the last decades. The basic idea of constraint propagation is to detect and remove inconsistent variable assignments that cannot participate in any feasible solution through the repeated analysis and evaluation of the variables, domains and constraints describing a specific problem instance. We describe efficient constraint propagation methods also known as consistency tests for the disjunctive scheduling problem (DSP) applications of which will be introduced in machine scheduling chap-

ters 8 to 10. We will further present and analyze both new and classical consistency tests involving a higher number of variables. They still can be implemented efficiently in a polynomial time. Further, the concepts of energetic reasoning and shaving are analyzed and discussed.

The other contribution is a classification of the consistency tests derived according to the domain reduction achieved. The particular strength of using consistency tests is based on their repeated application, so that the knowledge derived is propagated, i.e. reused for acquiring additional knowledge. The deduction of this knowledge can be described as the computation of a fixed point. Since this fixed point depends upon the order of the application of the tests, we first derive a necessary condition for its uniqueness. We then develop a concept of dominance which enables the comparison of different consistency tests as well as a method for proving dominance.

Chapter 17 is devoted to problems which perhaps closer reflect some specific features of scheduling in flexible manufacturing systems than other chapters do. Dynamic job shops are considered, i.e. such in which some events, particularly job arrivals, occur at unknown times. A heuristic for a static problem with mean tardiness as a criterion is described. It solves the problem at each time when necessary, and the solution is implemented on a rolling horizon basis. The next section deals with simultaneous assignment of machines and vehicles to jobs. This model is motivated by the production of helicopter parts in some factory. First we solve in polynomial time the problem of finding a feasible vehicle schedule for a given assignment of tasks to machines, and then present a dynamic programming algorithm for the general case. In the last section we are modeling manufacturing of acrylic-glass as a batch scheduling problem on parallel processing units under resource constraints. This section introduces the real world problem and reveals potential applications of some of the material in the previous chapters. In particular, a local search heuristic is described for constructing production sequences.

Chapter 18 serves two purposes. On one hand, we want to introduce a quite general solution approach for scheduling problems as they appear not only in manufacturing environments. On the other hand, we also want to survey results from interactive and knowledge-based scheduling which were not covered in this handbook so far. To combine both directions we introduce some broader aspects like computer integrated manufacturing and object-oriented modeling. The common goal of this chapter is to combine results from different areas to treat scheduling problems in order to answer quite practical questions. To achieve this we first concentrate on the relationship between the ideas of computer integrated manufacturing and the requirements concerning solutions of the scheduling problems. We present an object-oriented reference model which is used for the implementation of the solution approach. Based on this we discuss the outline of an intelligent production scheduling system using open loop interactive and closed loop knowledge-based problem solving. For reactive scheduling we suggest to use concepts from soft computing. Finally, we make some proposals concerning the integration of solution approaches discussed in the preceding chap-

ters with the ideas developed in this chapter. We use an example to clarify the approach of integrated problem solving and discuss the impact for computer integrated manufacturing.

Chapter 19 presents some examples of applying the scheduling theory for solving problems arising in logistics. Since logistics is strictly related with transportation, for the sake of completeness, we first present a short survey of various variants of the famous vehicle routing problem, which concerns designing routes for a fleet of vehicles to supply a set of customers. Since this problem is widely studied in the literature, we provide numerous references, which can direct the readers deeply interested in this field. Then we describe three problems arising in various modes of transportation, in overland, air and maritime transportation: the concrete delivery problem, the flight gate scheduling problem, and the berth and quay crane allocation problem, respectively. Based on these examples we show various aspects of incorporating scheduling theory in logistics. The concrete delivery problem is a vehicle routing problem in which schedules for concrete mixer vehicles are constructed to deliver concrete from depots to customers, taking into account the specificity of the perishable material being transported. On this example we show the process of modelling complex real world problems, providing the graph model and - based on it - the mixed integer programming model. Then we present a selected approach for the flight gate scheduling problem, concerning assigning aircrafts serving flights to airport gates. The described method is an example of transforming a scheduling problem to a related combinatorial problem (clique partitioning in this case), in order to utilize known algorithms to solve new problems. Finally, on the example of the berth and quay crane allocation problem, in which berths and cranes have to be assigned to ships for loading/unloading containers transported by them, we show a direct application of the scheduling models to solve this logistic case. For all three mentioned problems we provide a rich set of references for readers deeper interested in these and related subjects.

References

- Bak74 K. Baker, *Introduction to Sequencing and Scheduling*, J. Wiley, New York, 1974.
- BCSW86 J. Błażewicz, W. Cellary, R. Słowiński, J. Węglarz, *Scheduling under Resource Constraints: Deterministic Models*, J. C. Baltzer, Basel, 1986.
- Bru07 P. Brucker, *Scheduling Algorithms*, Springer, 5th ed., Berlin, 2007.
- BT09 K. R. Baker, D. Trietsch, *Principles of Sequencing and Scheduling*, J. Wiley, New Jersey, 2009.
- CCLL95 P. Chretienne, E. G. Coffman, J. K. Lenstra, Z. Liu (eds.), *Scheduling Theory and its Applications*, J. Wiley, New York, 1995.
- CMM67 R. W. Conway, W. L. Maxwell, L. W. Miller, *Theory of Scheduling*, Addison-Wesley, Reading, Mass., 1967.

- Cof76 E. G. Coffman, Jr. (ed.), *Scheduling in Computer and Job Shop Systems*, J. Wiley, New York, 1976.
- Eck77 K. Ecker, *Organisation von parallelen Prozessen*, BI-Wissenschaftsverlag, Mannheim, 1977.
- Fre82 S. French, *Sequencing and Scheduling: An Introduction to the Mathematics of the Job-Shop*, Horwood, Chichester, 1982.
- Gaw08 S. Gawiejnowicz, *Time-Dependent Scheduling*, Springer, Berlin-Heidelberg, 2008.
- GK87 S. K. Gupta, J. Kyparisis, Single machine scheduling research, *Omega-Int. J. Manage. Sci.* 15, 1987, 207-227.
- Len77 J. K. Lenstra, *Sequencing by Enumerative Methods*, Mathematical Centre Tract 69, Amsterdam, 1977.
- Leu04 J. Y.-T. Leung (ed.), *Handbook of Scheduling: Algorithms, Models and Performance Analysis*, Chapman & Hall/CRC, Boca Raton, 2004.
- LLR+93 E. L. Lawler, J. K. Lenstra, A. H. G. Rinnooy Kan, D. B. Shmoys, Sequencing and scheduling: Algorithms and complexity, in: S. C. Graves, A. H. G. Rinnooy Kan, P. H. Zipkin (eds.), *Handbook in Operations Research and Management Science, Vol. 4: Logistics of Production and Inventory*, Elsevier, Amsterdam, 1993.
- Pin16 M. Pinedo, *Scheduling: Theory, Algorithms, and Systems*, 5th ed., Springer, New York, 2016.
- Rin76 A. H. G. Rinnooy Kan, *Machine Scheduling Problems; Classification, Complexity and Computations*, Martinus Nijhoff, The Hague, 1976.
- RV09 Y. Robert, F. Vivien (eds.), *Introduction to Scheduling*, CRC Press, Boca Raton, 2009.
- TB06 V. T'kindt, J.-C. Billaut, *Multicriteria Scheduling: Theory, Models and Algorithms*, 2nd ed., Springer, Berlin, 2006.
- TGS94 V. S. Tanaev, V. S. Gordon, Y. M. Shafransky, *Scheduling Theory. Single-Stage Systems*, Kluwer, Dordrecht, 1994.
- TSS94 V. S. Tanaev, Y. N. Sotskov, V. A. Strusevich, *Scheduling Theory. Multi-Stage Systems*, Kluwer, Dordrecht, 1994.

2 Basics

In this chapter we provide the reader with basic notions used throughout the book. After a short introduction into sets and relations, decision problems, optimization problems and the encoding of problem instances are discussed. The way algorithms will be represented and problem membership of complexity classes are other essential issues which will be discussed. Afterwards graphs, especially certain types such as precedence graphs and networks that are important for scheduling problems, are presented. The last two sections deal with algorithmic methods used in scheduling such as enumerative algorithms (e. g. dynamic programming and branch and bound) and heuristic approaches (e. g. tabu search, simulated annealing, ejection chains, and genetic algorithms).

2.1 Sets and Relations

Sets are understood to be any collection of distinguishable objects, such as the set $\{1, 2, \dots\}$ of natural numbers, denoted by $\mathbb{N}$, the set $\mathbb{N}_0$ of non-negative integers, the set of real numbers, $\mathbb{R}$, or the set of non-negative reals $\mathbb{R}_{\geq 0}$. Given real numbers a and b , $a \leq b$, then $[a, b]$ denotes the *closed interval* from a to b , i.e. the set of reals $\{x \mid a \leq x \leq b\}$. *Open intervals* $((a, b) := \{x \mid a < x < b\})$ and *half open intervals* are defined similarly.

In scheduling theory we are normally concerned with finite sets; so, unless infinity is stated explicitly, the sets are assumed to be finite.

For set S , $|S|$ denotes its *cardinality*. The *power set* of S (i.e. the set of all subsets of S) is denoted by $\mathcal{P}(S)$. For an integer k , $0 \leq k \leq |S|$, the set of all subsets of cardinality k is denoted by $\mathcal{P}_k(S)$.

The *Cartesian product* $S_1 \times \dots \times S_k$ of sets $S_1, \dots, S_k$ is the set of all tuples of the form $(s_1, s_2, \dots, s_k)$ where $s_i \in S_i$, $i = 1, \dots, k$, i.e. $S_1 \times \dots \times S_k = \{(s_1, \dots, s_k) \mid s_i \in S_i, i = 1, \dots, k\}$. The k -fold Cartesian product $S \times \dots \times S$ is denoted by S^k .

Given sets $S_1, \dots, S_k$, a subset Q of $S_1 \times \dots \times S_k$ is called a *relation over* $S_1, \dots, S_k$. In the case $k = 2$, Q is called a *binary relation*. For a binary relation Q over S_1 and S_2 , the sets S_1 and S_2 are called *domain* and *range*, respectively. If Q is a relation over $S_1, \dots, S_k$, with $S_1 = \dots = S_k = S$, then we simply say: Q is a (k -*ary*) *relation over* S . For example, the set of edges of a directed graph (see Section 2.3) is a binary relation over the vertices of the graph.

Let S be a set, and Q be a binary relation over S . Then, $Q^{-1} = \{(a, b) \mid (b, a) \in Q\}$ is the *inverse* to Q . Relation Q is *symmetric* if $(a, b) \in Q$ implies $(b, a) \in Q$. Q is *antisymmetric* if for $a \neq b$, $(a, b) \in Q$ implies $(b, a) \notin Q$. Q is *reflexive* if $(a, a) \in Q$ for all $a \in S$. Q is *irreflexive* if $(a, a) \notin Q$ for all $a \in S$. Q is *transitive* if for all $a, b, c \in S$, $(a, b) \in Q$ and $(b, c) \in Q$ implies $(a, c) \in Q$.

A binary relation over S represents a *partial order*. A set with a binary relation is called a *partially ordered set* or *poset* if and only if it is reflexive, anti-symmetric and transitive. A binary relation over S is called an *equivalence relation* (over S) if it is reflexive, symmetric, and transitive.

Given set J of n closed intervals of reals, $J = \{I_i \mid I_i = [a_i, b_i], a_i \leq b_i, i = 1, \dots, n\}$, a partial order $\leq_J$ on J can be defined by

$$I_i \leq_J I_j \Leftrightarrow (I_i = I_j) \text{ or } (b_i \leq a_j), i, j \in \{1, \dots, n\}.$$

A poset Q over S is called an *interval order* if and only if there exists a bijection from S to a set of intervals, $s_i \rightarrow I_i = [a_i, b_i]$, such that for any $s_i, s_j \in S$ we have $(s_i, s_j) \in Q$ exactly when $b_i < a_j$.

Let $I = (n_1, \dots, n_k)$ and $I' = (n'_1, \dots, n'_{k'})$ be sequences of integers, and $k, k' \geq 0$. If $k = 0$ then I is the empty sequence. We say that I is *lexicographically smaller* than I' , written $I \triangleleft I'$, if

- (i) the two sequences agree up to some index j , but $n_{j+1} < n'_{j+1}$ (i.e. there exists j , $0 \leq j \leq k$, such that for all i , $1 \leq i \leq j$, $n_i = n'_i$ and $n_{j+1} < n'_{j+1}$), or if
- (ii) sequence I is shorter, and the two sequences agree up to the length of I (i.e. $k < k'$ and $n_i = n'_i$ for all i , $1 \leq i \leq k$).

Let Q and $\mathcal{P}$ be binary relations over set S . Then the *relational product* $Q \circ \mathcal{P}$, defined as $\{(a, b) \mid \exists x \in S, (a, x) \in Q, (x, b) \in \mathcal{P}\}$, is a relation over S . Generally, we write Q^0 for $\{(a, a) \mid a \in S\}$, $Q^1 = Q$, and $Q^{i+1} = Q^i \circ Q$ for $i > 1$. The union $Q^* = \cup \{Q^i \mid i \geq 0\}$ is called the *transitive closure* of Q .

A *function* from $\mathcal{A}$ to $\mathcal{B}$ ($\mathcal{A} \rightarrow \mathcal{B}$; $\mathcal{A}$ and $\mathcal{B}$ are not necessarily finite) is a relation F over $\mathcal{A}$ and $\mathcal{B}$ such that for each $a \in \mathcal{A}$ there exists just one $b \in \mathcal{B}$ for which $(a, b) \in F$; instead of $(a, b) \in F$ we usually write $F(a) = b$. Set $\mathcal{A}$ is called the *domain* of F and set $\{b \mid b \in \mathcal{B}, \exists a \in \mathcal{A}, (a, b) \in F\}$ is called the *range* of F . F is called *surjective*, or *onto* $\mathcal{B}$ if for each element $b \in \mathcal{B}$ there is at least one element $a \in \mathcal{A}$ such that $F(a) = b$. Function F is said to be *injective*, or *one-one* if for each pair of elements, $a_1, a_2 \in \mathcal{A}$, $F(a_1) = F(a_2)$ implies $a_1 = a_2$. A function that is both surjective and injective is called *bijective*. A bijective function $F: \mathcal{A} \rightarrow \mathcal{A}$ is called a *permutation* of $\mathcal{A}$. Though we are able to represent functions in

special cases by means of tables we usually specify functions in a more or less abbreviated way that specifies how the function values are to be determined. For example, for $n \in \mathbb{N}$, the factorial function $n!$ denotes the set of pairs $\{(n, m) \mid n \in \mathbb{N}, m = n \cdot (n-1) \cdots 3 \cdot 2\}$. Other examples of functions are polynomials, exponential functions and logarithms.

We will say that function $f: \mathbb{N} \rightarrow \mathbb{R}^+$ is of order g , written $O(g(k))$, if there exist constants c and $k_0 \in \mathbb{N}$ such that $f(k) \leq cg(k)$ for all $k \geq k_0$.

2.2 Problems, Algorithms, Complexity

2.2.1 Problems and Their Encoding

In general, the scheduling problems we consider belong to a broader class of combinatorial search problems. A *combinatorial search problem* Π is a set of pairs (I, A) , where I is called an *instance* of a problem, i.e. a finite set of *parameters* (understood generally, e.g. numbers, sets, functions, graphs) with specified values, and A is an *answer (solution)* to the instance. As an example of a search problem let us consider *merging* two sorted sequences of real numbers. Any instance of this problem consists of two finite sequences of reals e and f sorted in non-decreasing order. The answer is the sequence g consisting of all the elements of e and f arranged in non-decreasing order.

Let us note that among search problems one may also distinguish two subclasses: optimization and decision problems. An *optimization problem* is defined in such a way that an answer to its instance specifies a solution for which a value of a certain objective function is at its optimum (an *optimal solution*). On the other hand, an answer to an instance of a *decision problem* may take only two values, either "yes" or "no". It is not hard to see, that for any optimization problem, there always exists a decision counterpart, in which we ask (in the case of minimization) if there exists a solution with the value of the objective function less than or equal to some additionally given threshold value y . (If in the basic problem the objective function has to be maximized, we ask if there exists a solution with the value of the objective function $\geq y$.) The following example clarifies these notions.

Example 2.2.1 Let us consider an optimization *knapsack problem*.

Knapsack

Instance: A finite set of elements $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$, each of which has an integer weight $w(a_i)$ and value $v(a_i)$, and an integer capacity b of a knapsack.

Answer: Subset $\mathcal{A}' \subseteq \mathcal{A}$ for which $\sum_{a_i \in \mathcal{A}'} v(a_i)$ is at its maximum, subject to the constraint $\sum_{a_i \in \mathcal{A}'} w(a_i) \leq b$ (i.e. the total value of chosen elements is at its maximum and the sum of weights of these elements does not exceed knapsack capacity b).

The corresponding decision problem is denoted as follows. (To distinguish optimization problems from decision problems the latter will be denoted using capital letters.)

KNAPSACK

Instance: A finite set of elements $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$, each of which has an integer weight $w(a_i)$ and value $v(a_i)$, an integer knapsack capacity b and threshold value y .

Answer: "Yes" if there exists subset $\mathcal{A}' \subseteq \mathcal{A}$ such that

$$\sum_{a_i \in \mathcal{A}'} v(a_i) \geq y \text{ and } \sum_{a_i \in \mathcal{A}'} w(a_i) \leq b.$$

Otherwise "No". □

When considering search problems, especially in the context of their solution by computer algorithms, one of the most important issues that arises is a question of data structures used to encode problems. Usually to encode instance I of problem Π (that is particular values of parameters of problem Π) one uses a finite *string* of symbols $x(I)$. These symbols belong to a predefined finite set Σ (usually called an *alphabet*) and the way of coding instances is given as a set of encoding rules (called *encoding scheme* e). By *input length* (*input size*) $|I|$ of instance I we mean here the length of string $x(I)$. Let us note that the requirement that an instance of a problem is encoded by a finite string of symbols is the only constraint imposed on the class of search problems which we consider here. However, it is rather a theoretical constraint, since we will try to characterize algorithms and problems from the viewpoint of the application of real computers.

Now the encoding scheme and its underlying data structure is defined in a more precise way. For representation of mathematical objects we use set Σ that contains the usual characters, i.e. capital and small Arabic letters, capital and small Greek letters, digits $(0, \dots, 9)$, symbols for mathematical operations such as $+$, $-$, $\times$, $/$, and various types of parentheses and separators. The class of mathematical objects, $\mathcal{A}$, is then mapped to the set Σ^* of words over the alphabet Σ by means of a function $\rho: \mathcal{A} \rightarrow \Sigma^*$, where Σ^* denotes the set of all finite strings (words) made up of symbols belonging to Σ . Each mathematical object $A \in \mathcal{A}$ is represented as a *structured string* in the following sense: Integers are represented by their decimal representation. A square matrix of dimension n with integer el-

ements will be represented as a finite list whose first component represents matrix dimension n , and the following n^2 components represent the integer matrix elements in some specific order. For example, the list is a structured string of the form $(n, a(1, 1), \dots, a(1, n), a(2, 1), \dots, a(2, n), \dots, a(n, n))$ where n and all the $a(i, j)$ are structured strings representing integers. The length of encoding (i.e. the complexity of storing) an integer k would then be of order $\log k$, and that of a matrix would be of order $n^2 \log k$ where k is an upper bound for the absolute value of each matrix element. Real numbers will be represented either in decimal notation (e.g. 3.14159) or in half-logarithmic representation using mantissa and exponent (e.g. $0.314159 \cdot 10^1$). Functions may be represented by tables which specify the function (range) value for each domain value. Representations of more complicated objects (e.g. graphs) will be introduced later, together with the definition of these types of objects.

As an example let us consider encoding of a particular instance of the knapsack problem defined in Example 2.2.1. Let the number n of elements be equal to 6 and let an encoding scheme define values of parameters in the following order: n , weights of elements, values of elements, knapsack's capacity b . A string coding an exemplary instance is: 6, 4, 2, 12, 15, 3, 7, 1, 4, 8, 12, 5, 7, 28.

The above remarks do not exclude the usage of any other *reasonable encoding scheme* which does not cause an exponential growth of the input length as compared with other encoding schemes. For this reason one has to exclude unary encoding in which each integer k is represented as a string of k 1's. We see that the length of encoding this integer would be k which is exponentially larger, as compared to the above decimal encoding.

In practice, it is worthwhile to express the input length of an instance as a function depending on the number of elements of some set whose cardinality is dominating for that instance. For the knapsack problem defined in Example 2.2.1 this would be the number of elements n , for the merging problem - the total number of elements in the two sequences, for the scheduling problem - the number of tasks. This assumption, usually made, in most cases reduces practically to the assumption that a computer word is large enough to contain any of the binary encoded numbers comprising an instance. However, in some problems, for example those in which graphs are involved, taking as input size the number of nodes may appear too great a simplification since the number of edges in a graph may be equal to $n(n-1)/2$. Nevertheless, in practice one often makes this simplification to unify computational results. Let us note that this simplification causes no exponential growth of input length.

2.2.2 Algorithms

Let us now pass to the notion of an algorithm and its complexity function. An algorithm is any finite procedure for solving a problem (i.e. for giving an answer). We will say that an algorithm *solves search problem* Π , if it finds a solution for

any instance I of Π . In order to keep the representation of algorithms easily understandable we follow a structural approach that uses language concepts known from structural programming, such as case statements, or loops of various kinds. Like functions or procedures, algorithms may also be called in an algorithm. Parameters may be used to import data to or export data from the algorithm. Besides these, we also use mathematical notations such as set-theoretic notations.

In general, an algorithm consists of two parts: a *head* and a *method*. The head starts with the keyword **Algorithm**, followed by an identifying number and, optionally, a descriptor (a name or a description of the purpose of the algorithm) and a reference to the author(s) of the algorithm. Input and output parameters are omitted in cases where they are clear from the context. In other cases, they are specified as a parameter list. In even more complex cases, two fields, *Input (Instance)*: and *Output (Answer)*: are used to describe parameters, and a field *Method*: is used to describe the main idea of the algorithm. As in PASCAL, a block is embraced by **begin** and **end**. Each block is considered as a sequence of instructions. An instruction itself may again be a block, an assignment-, an else-, or a case- operation, or a loop (**for**, **while**, **repeat** ... **until**, or a general **loop**), a **call** of another algorithm, or an exit instruction to terminate a loop instruction (**exit loop**, etc.) or the algorithm or procedure (just **exit**). The right hand side of an assignment operation may be any mathematical expression, or a function call. Case statements partition actions of the algorithm into several branches, depending on the value of a control variable. Loop statements may contain formulations such as: "**for all** $a \in \mathcal{M}$ **do** ..." or "**while** $\mathcal{M} \neq \emptyset$ **do** ...". If a loop is preempted by an exit statement the algorithm jumps to the first statement after the loop. Comments are started with two minus signs and are finished at the end of the line. If a comment needs more than one line, each comment line starts with '--'.

Algorithms should reflect the main idea of the method. Details like output layouts are omitted. Names for procedures, functions, variables etc. are chosen so that they reflect the semantics behind them. As an example let us consider an algorithm solving the problem of merging two sequences as defined at the beginning of this section.

Algorithm 2.2.2 *merge*.

Input: Two sequences of reals, $e = (e[1], \dots, e[n])$ and $f = (f[1], \dots, f[m])$, both sorted in non-decreasing order.

Output: Sequence $g = (g[1], \dots, g[n+m])$ in which all elements are arranged in non-decreasing order.

begin

$i := 1; j := 1; k := 1; \quad \text{-- initialization of counters}$

while $(i \leq n)$ **and** $(j \leq m)$ **do**

-- the while loop merges elements of sequences e and f into g ;

-- the loop is executed until all elements of one of the sequences are merged